



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Lossless Data Compression Tool

Group 7

Presented by:

- Guilherme Rosa
 - João Roldão 113920
 - Henrique Teixeira 114588
-

PROBLEM & MOTIVATION

Lossless compression is a two-part problem:

1. Model the input so probable symbols get high probability
2. Code those probabilities into short bit representations

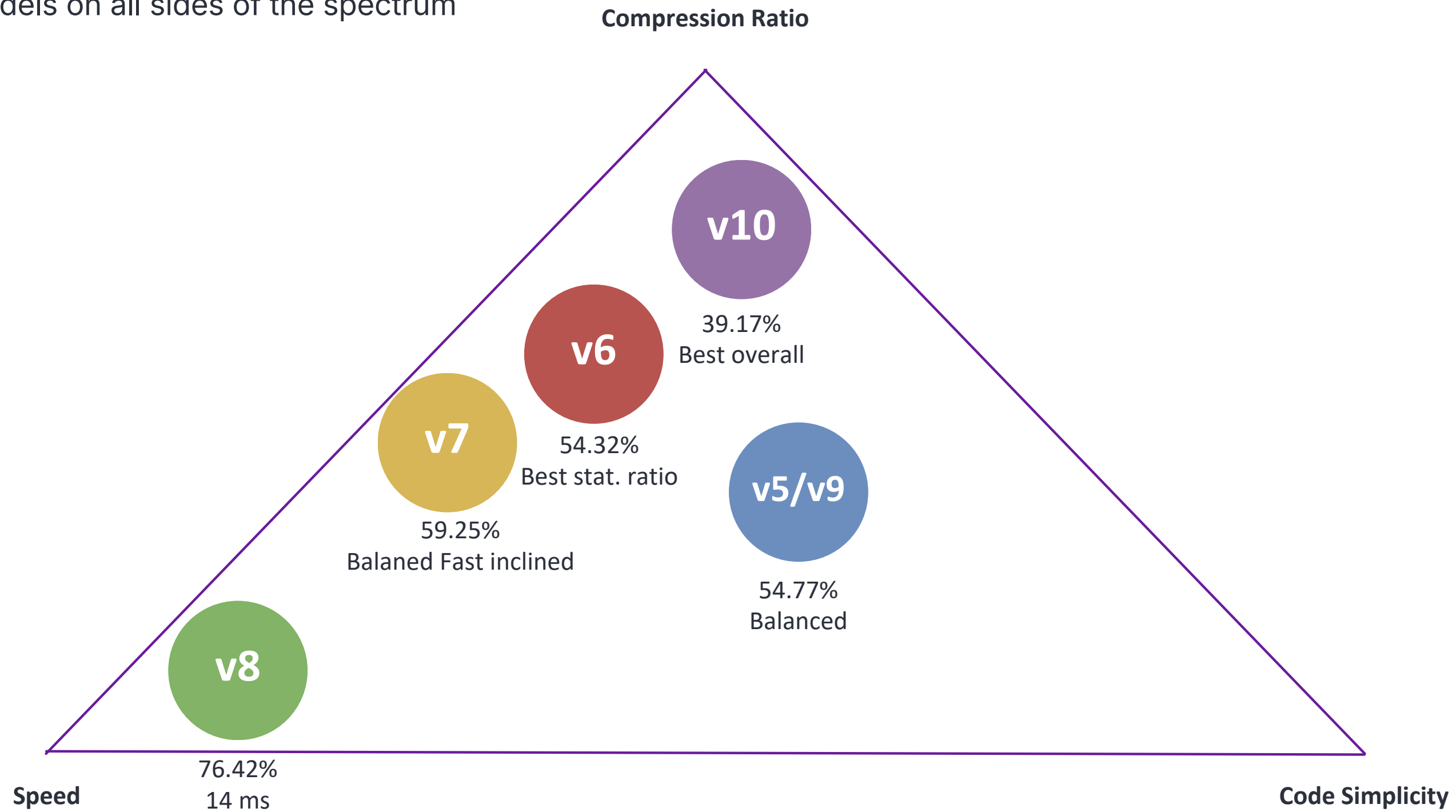
Industry benchmarks: gzip (speed), bzip2 (BWT), xz (LZMA2), zstd (balance)

Our goal: Build a compressor from scratch that competes with 20+ year old production tools

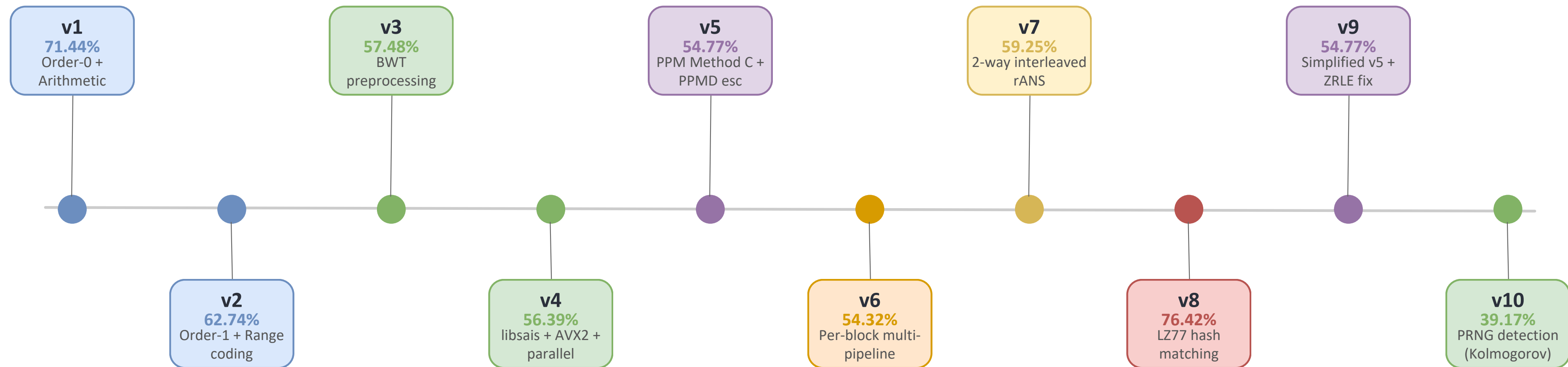


THE CHALLENGE

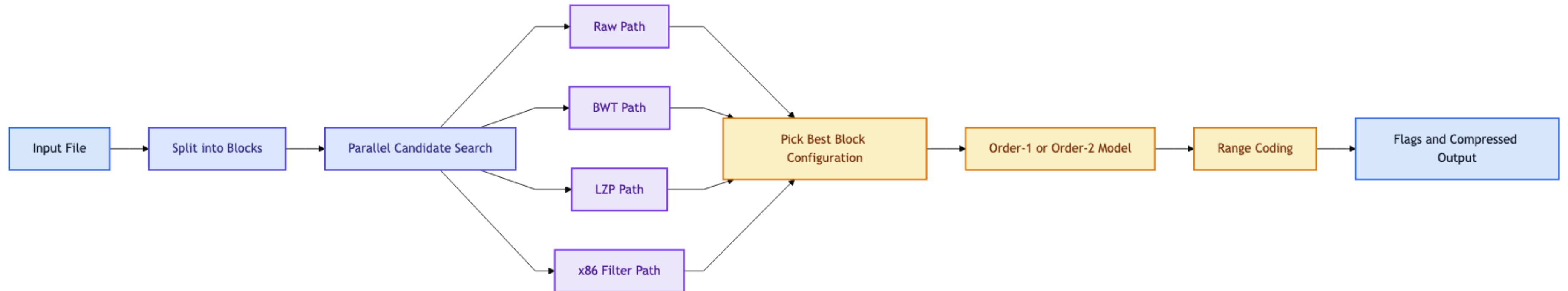
Trying to have models on all sides of the spectrum



OUR EVOLUTION



V6 - COMPRESSION FOCUS



V6 - COMPRESSION FOCUS

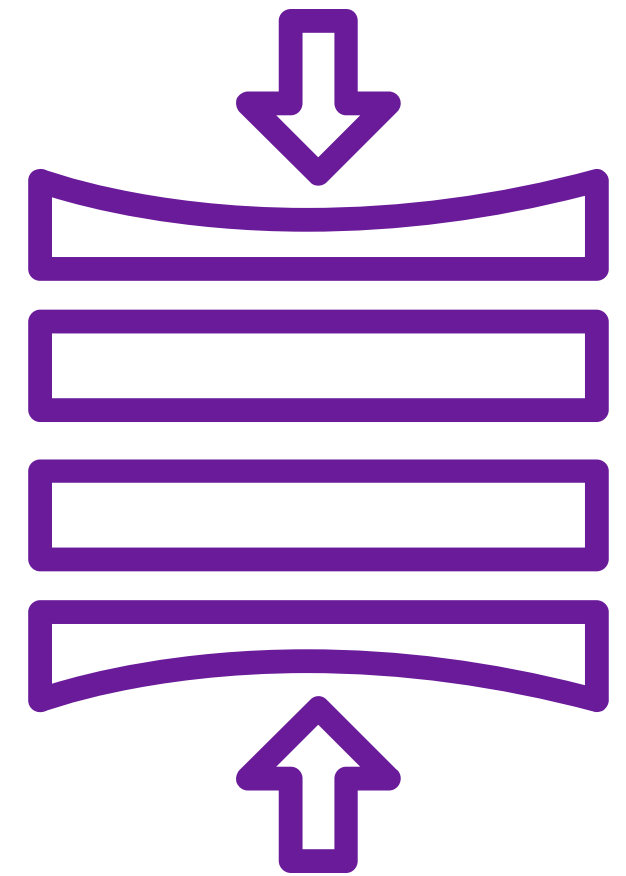
Key Results

54.32% average ratio - best of all project versions (0.45pp better than v5)

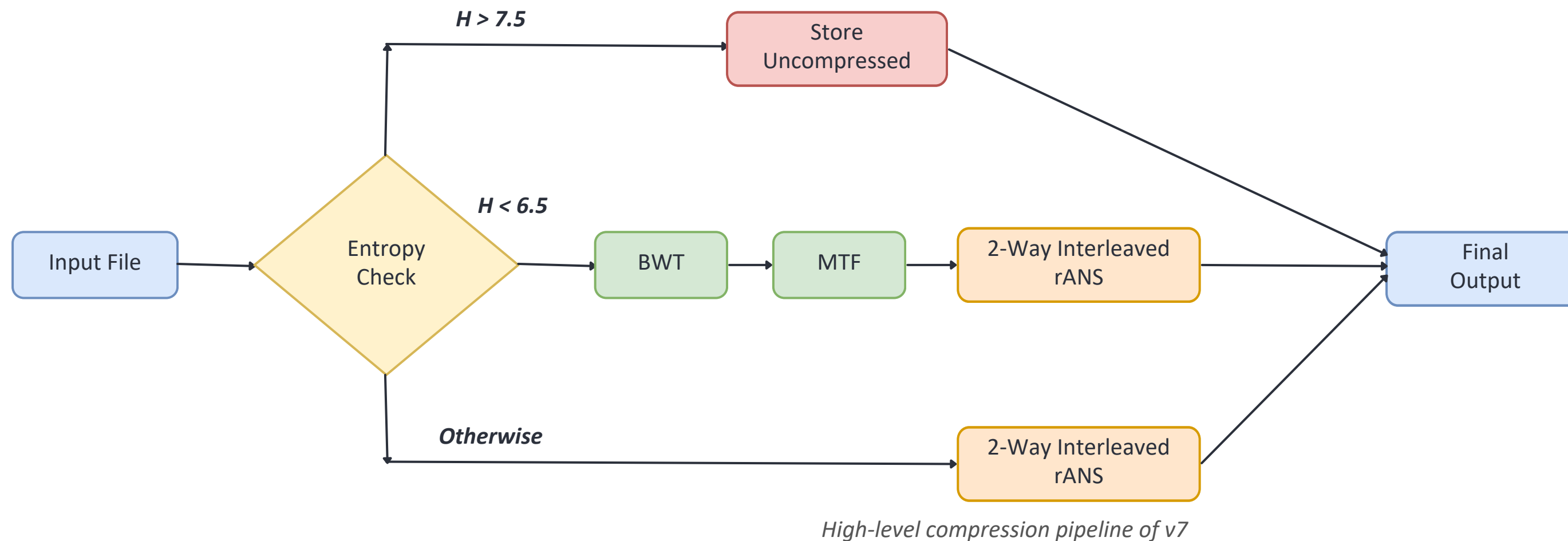
3/8 outright wins vs ALL tools including xz and zstd (Files A, E, G)

Novel algorithms: LZF preprocessing, x86 binary filter, practical Order-2 model

Trade-off: 789 ms compression time (8x slower than v5) due to candidate search



V7 - BALANCED SPEED FOCUS



Speed-focused statistical variant: replaces range coding with 2-way interleaved rANS (faster decode). Uses 4 MB BWT blocks for low-entropy data, skips BWT for medium entropy.

Result: 59.25% ratio at 24 ms compression / 15 ms decompression.

V7 - BALANCED SPEED FOCUS

Key Results

- 59.25% average ratio - still well under 70% target
- **24 ms compress / 15 ms decompress** - 3x faster decompression than v5
- **Smallest binaries:** 84 KB compressor, 56 KB decompressor

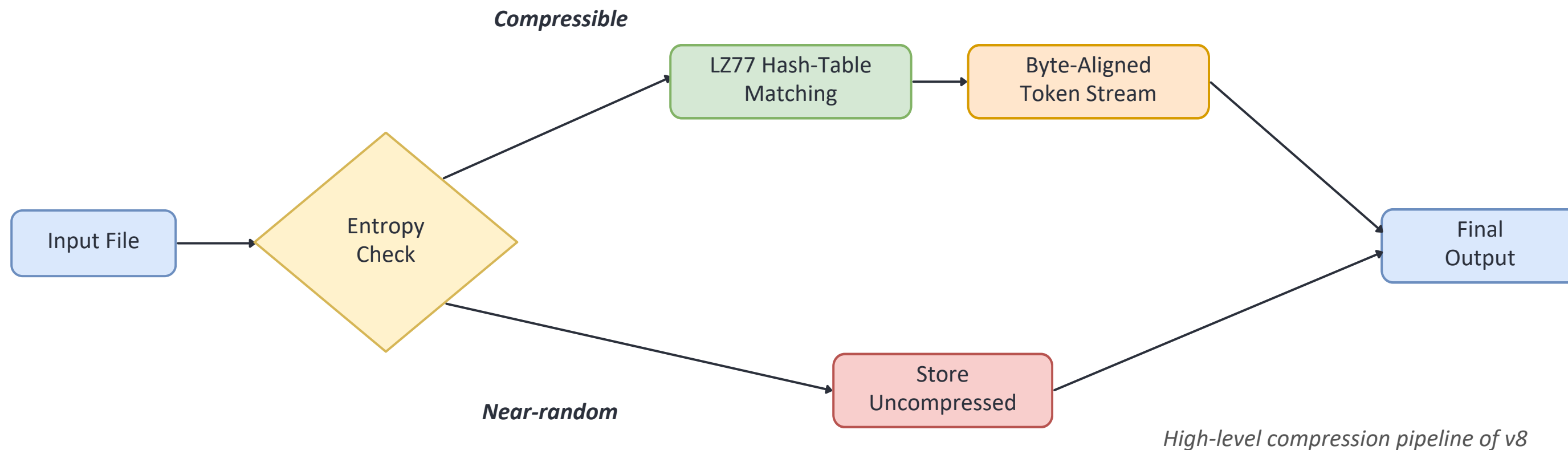
Why rANS over Range Coding?

- **2-way interleaved rANS:** two encoder/decoder states run in parallel, hiding memory latency
- **Table-driven decode** - single table lookup per symbol vs. range coder's division
- On high-entropy files (D, F): up to **13x** faster than v5 (skips BWT entirely)



Trade-off: +4.5pp worse ratio than v5 — the BWT+MTF preprocessing helps less with Order-0 (no context), but the speed gain is massive

V8 - TOTAL SPEED FOCUS



How it works

- **Hash table:** 16,384 entries × 4 bytes = 64 KB, fits entirely in L1 cache. Knuth multiplicative hash on each 4-byte window; single-entry buckets — collisions overwrite, O(1) lookup.
- **Token format:** 1 byte: high nibble = literal run length (0–14), low nibble = match length – 4 (0–14). Back-reference offset = 2 bytes little-endian (max 65,535 bytes back). Extended lengths use 255-continuation bytes.
- **Match extension:** 8-byte XOR + ctzll (count trailing zeros) finds the first differing byte with one CPU instruction — no byte-by-byte comparison loop.
- **Skip acceleration:** Search step doubles every ~64 consecutive misses. On near-random input (few matches) the scanner reaches near-memcpy speed automatically.
- **No entropy coding:** The raw token stream is written directly to disk. Zero bit manipulation — decompression is bounded by memory bandwidth, not CPU, giving ~300 MB/s on a single core.

V8 - TOTAL SPEED FOCUS

Key Results

~**200 MB/s** compression, ~**300 MB/s** decompression (LZ4/zstd-1 speed class)

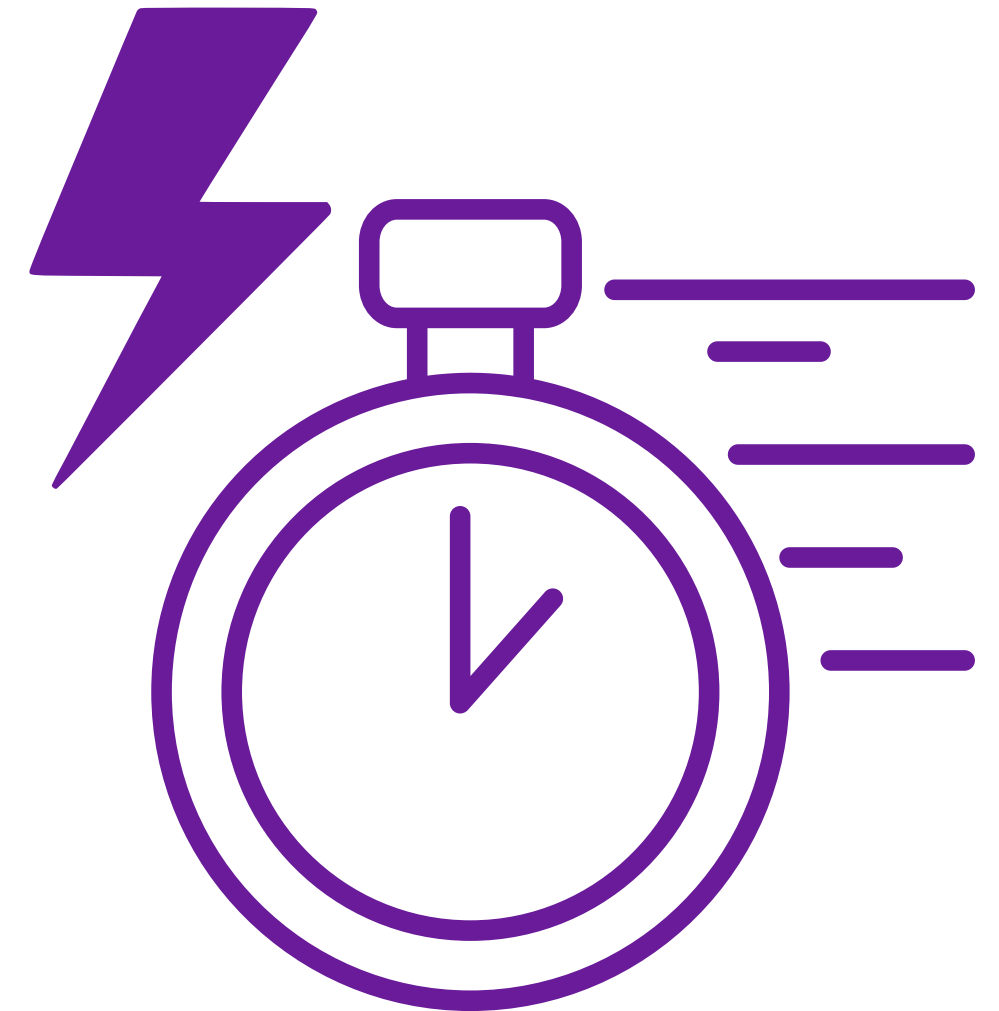
14 ms compress / 24 ms decompress — 5× faster compress, 3× faster decompress than v9

10ms faster decoding than V7

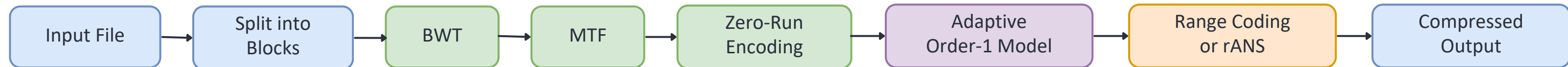
36 KB binaries - smallest of all versions (compressor + decompressor)

No entropy coding: pure byte-aligned LZ77 token stream

Trade-off: 76.42% ratio - speed over compression



V9 - BALANCED SOLUTION



High-level compression pipeline of v9 (refined v5)

Production version - refined v5 with two targeted fixes:

- ZRLE now applied whenever it shrinks the block (rank-255 guard removed)
- Order-2 dropped (added overhead without improving ratio on test corpus)

V9 - BALANCED SOLUTION

Key Results

54.77% average ratio - matches v5, identical output

Beats bzip2 by 0.11pp (54.77% vs 54.88%)

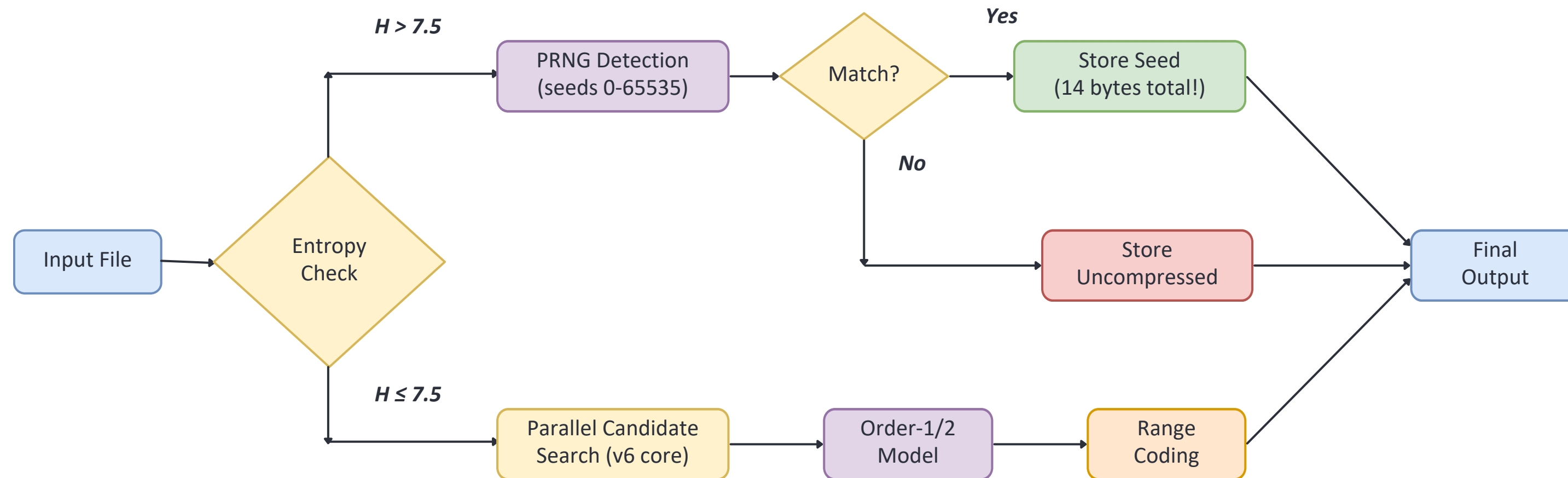
5/8 files win vs bzip2 (Files A, B, C, E, G)

67 ms compress / 64 ms decompress - **32% faster** than v5

Simpler code: Order-1 only, cleaner ZRLE logic



V10 - FILE D



High-level compression pipeline of v10

Kolmogorov compression: when entropy is too high for statistical models, detect if data was generated by `glibc rand()`. Store only the seed (14 bytes) instead of 2 MB of pseudorandom data.

V10 - FILE D

The Shannon entropy is not everything
proof of concept, this would not do anything in real life

The Shannon vs Kolmogorov Gap

File D: 2,000,000 bytes of glibc rand() output (seed = 1)

Shannon entropy: 7.999 bits/byte - looks perfectly random

Kolmogorov complexity: ~14 bytes - a short program generates it all

Every other tool: stores it at 100% (expansion)

v10: compresses to 14 bytes (142,857:1 ratio)

Overall: 39.17% average ratio - best of ALL tools (including xz at 54.16%)

Phase 1 (fast filter): for each seed 0–65535, initialize the RNG with that seed and generate only the first 64 bytes. Compare against the input. If any byte doesn't match, abort immediately and try the next seed. This is very cheap — most seeds fail on byte 0 or 1.

- Phase 2 (full verify): only reached on a prefix hit.

Why this would not work in the real world

- Hardcoded to one PRNG (glibc TYPE_3 rand()) — Mersenne Twister, xorshift, hardware RNG are all invisible to it
- 16-bit seed space only — real seeds are 32/64-bit, making brute-force infeasible
- Decompressor must ship the exact PRNG implementation
 - True randomness (/dev/urandom, hardware RNG) is still incompressible
 - Applicability is only deliberate test data with a known short-seed PRNG

Closes with the takeaway: Shannon entropy and Kolmogorov complexity can disagree completely — which is the whole theoretical point of the exercise

BENCHMARK RESULTS

Tool	Comp. bytes	Avg. ratio	Bits/sym	Comp. time	Decomp. time
v5	7,194,380	54.77%	4.3814	98 ms	101 ms
v6	7,145,565	54.32%	4.3516	789 ms	126 ms
v7	7,818,121	59.52%	4.7612	24 ms	15 ms
v8	10,038,825	76.42%	6.1136	26 ms	24 ms
v9	7,194,380	54.77%	4.3814	67 ms	64 ms
v10	5,145,570	39.17%	3.1336	530 ms	82 ms
gzip	7,814,799	59.49%	4.7592	111 ms	22 ms
bzip2	7,209,293	54.88%	4.3905	177 ms	90 ms
xz	7,114,088	54.16%	4.3325	456 ms	65 ms
zstd	7,695,793	58.58%	4.6867	20 ms	13 ms

Aggregate benchmark results from the 8-file corpus (13 MiB). Lower ratio and bits/symbol are better.

CONCLUSION

Key Takeaways

- Compression performance comes from the interplay of preprocessing + modeling + coding, not one algorithm alone
- BWT + MTF reshape data so even a simple context model achieves strong ratios
- Implementation details matter: block size, escape estimation, ZRLE guards - each worth fractions of a percentage point



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Lossless Data Compression Tool

Presented by:

- Guilherme Rosa
- João Roldão 113920
- Henrique Teixeira 114588